

JavaScript Inheritance Types

Assignment 1 - Full-Stack Development Lab

Single

Multilevel

Hierarchical

Multiple (Mixin)

Hybrid

Anil Neerukonda Institute of Technology & Sciences (ANITS)

Permanently Affiliated to Andhra University | Approved by AICTE | Autonomous

Programming Language: JavaScript (ES6+ Classes)

One parent class inherits to exactly one child class.

```
Animal (Parent)
+-- Dog extends Animal (Child)
```

OUTPUT

> Parent class (Animal):

```
speak()           -> Cat says "Meow"
breathe()         -> Cat breathes air.
```

> Child class (Dog extends Animal):

```
speak() [overridden] -> Rex barks loudly: "Woof!"
breathe() [inherited] -> Rex breathes air.
fetch() [own method]  -> Rex (German Shepherd) fetches the ball!
instanceof Animal     -> true
instanceof Dog         -> true
```

Source: *Assignments/Assignment_1/src/1_single_inheritance.js*

A chain: Person -> Employee -> Manager. Each level extends the one above.

```
Person      (Level 0 - GrandParent)
  +-- Employee extends Person      (Level 1)
      +-- Manager extends Employee (Level 2)
```

OUTPUT

> Level 0 - Person:

```
introduce()      -> Hi, I am Alice, 25 years old.
eat()            -> Alice is eating.
```

> Level 1 - Employee (extends Person):

```
introduce() [inherited]      -> Hi, I am Bob, 30 years old.
work()       [own]            -> Bob works at TCS.
getSalary()  [own]            -> Bob's salary: Rs.8,00,000
```

> Level 2 - Manager (extends Employee):

```
introduce() [overridden]      -> Hi, I am Carol, Manager at Infosys.
eat()       [from Person 2^]   -> Carol is eating.
work()      [from Employee]    -> Carol works at Infosys.
getSalary() [from Employee]    -> Carol's salary: Rs.20,00,000
manage()    [own]              -> Carol manages a team of 15 at Infosys.
instanceof Person              -> true
instanceof Employee            -> true
instanceof Manager             -> true
```

Source: `Assignments/Assignment_1/src/2_multilevel_inheritance.js`

One parent class - multiple independent child classes inherit from it.

```
Vehicle (Parent)
+-- Car    extends Vehicle
+-- Bike   extends Vehicle
+-- Truck  extends Vehicle
```

OUTPUT

> Car (extends Vehicle):

```
describe()          -> Car: Toyota, 4 doors, 180 km/h
move() [inherited]  -> Toyota is moving at 180 km/h.
honk() [own]        -> Toyota car honks: Beep Beep!
```

> Bike (extends Vehicle):

```
describe()          -> Bike: Royal Enfield (Cruiser), 140 km/h
move() [inherited]  -> Royal Enfield is moving at 140 km/h.
wheelie() [own]     -> Royal Enfield Cruiser bike does a wheelie!
```

> Truck (extends Vehicle):

```
describe()          -> Truck: Tata, Payload: 20T, 100 km/h
move() [inherited]  -> Tata is moving at 100 km/h.
loadCargo() [own]   -> Tata truck loads 20 tons of cargo.
car instanceof Vehicle -> true
bike instanceof Vehicle -> true
car instanceof Bike   -> false
```

Source: `Assignments/Assignment_1/src/3_hierarchical_inheritance.js`

Multiple Inheritance (Mixin Pattern)

JS has no native MI. Mixin functions compose multiple behaviours onto one class.

```
Mixins: Flyable ? Swimmable ? Runnable

Duck  extends Runnable( Swimmable( Flyable( AnimalBase ) ) )
Eagle extends Runnable( Flyable( AnimalBase ) )
Fish  extends Swimmable( AnimalBase )
```

OUTPUT

> Duck (Flyable + Swimmable + Runnable):

```
describe()      -> Donald is a Duck.
fly()           -> Donald spreads wings and soars!
swim()          -> Donald dives in and swims!
run()           -> Donald runs at full speed!
quack()         -> Donald quacks: Quack Quack!
```

> Eagle (Flyable + Runnable):

```
describe()      -> Bald Eagle is a Eagle.
fly()           -> Bald Eagle spreads wings and soars!
hunt()          -> Bald Eagle hunts prey from above!
'swim' in eagle? -> false
```

> Fish (Swimmable only):

```
describe()      -> Nemo is a Fish.
swim()          -> Nemo dives in and swims!
breatheUnderwater() -> Nemo breathes through gills.
'fly' in fish?   -> false
duck instanceof AnimalBase -> true
```

Source: `Assignments/Assignment_1/src/4_multiple_inheritance__inheritance.js`

Hierarchical + Multilevel + Mixin combined in one class hierarchy.

```
Shape  (Root)
  +-- TwoDShape  extends Shape
    |  +-- Circle    extends TwoDShape
    |  +-- Rectangle extends TwoDShape
  +-- ThreeDShape extends Shape
        +-- Sphere  extends Colorable( ThreeDShape ) [+mixin]
```

OUTPUT

> Circle (Shape -> TwoDShape -> Circle):

```
info()          -> Circle | 2D | Area=153.94 | Perimeter=43.98
instanceof Shape -> true
instanceof TwoDShape -> true
```

> Rectangle (Shape -> TwoDShape -> Rectangle):

```
info()          -> Rectangle | 2D | Area=50.00 | Perimeter=30.00
instanceof TwoDShape -> true
```

> Sphere (Shape -> ThreeDShape -> Sphere + Colorable):

```
info()          -> Sphere | 3D | Area=201.06 | Volume=268.08
getColor()      -> Color: Blue
setColor('Gold') -> Sphere color set to Gold.
getColor() after set -> Color: Gold
instanceof ThreeDShape -> true
'setColor' in sphere -> true
```

Source: `Assignments/Assignment_1/src/5_hybrid_inheritance.js`

Summary: JavaScript Inheritance Types

Type	Description	Classes	JS Support
Single	1 parent -> 1 child	Animal -> Dog	Native extends
Multilevel	Chain A -> B -> C	Person -> Employee -> Manager	Native extends
Hierarchical	1 parent -> many children	Vehicle -> Car/Bike/Truck	Native extends
Multiple (Mixin)	Many parents -> 1 child	Flyable+Swimmable -> Duck	Mixin pattern
Hybrid	Combo of 2+ types	Shape -> 2D/3D + Mixin	extends + Mixin

Key Concepts Used

class / extends	ES6 syntax for defining and inheriting classes
super()	Calls the parent class constructor or method
Method Overriding	Child redefines a method inherited from parent
instanceof	Checks if an object belongs to a class hierarchy
Mixin Pattern	Composable function that adds behaviour to any base class
Prototype Chain	JS resolves methods up the chain at runtime